

EE147 Assignment #3 Questions

On Bender, compare the execution time of a 1024×1024 square matrix multiplication compared to a 4096×256 and 256×4096 rectangular matrix multiply. All input matrices have 1M entries. Which is faster? Can you explain the observed behavior?

- 1) The 1024×1024 square matrix is about twice the speed of the 4096×256 times 256×4096 matrix. This behavior can be explained because the memory access patterns are better on the square matrix than the arbitrary matrix size. In the square matrix, the values can be frequently reused in shared memory. This speeds up the calculation as new values do not need to be reloaded frequently. However, the arbitrary matrix frequently reloads new data into shared memory, taking time away from calculating the matrix multiplication.

Let's now vary the size of the thread blocks in your matrix multiplication implementation. Reasonable thread block size can be 8×8 , 16×16 , or 32×32 . Report the execution time for each. Which tile size performed the fastest, which tile size performed the slowest? Why do you think that is?

- 2) A 16×16 block size worked the best for me. This utilizes the balance between parallelism and resource allocation. This block size does not exceed the hardware's limitations, resulting in efficient usage and the best calculation time. The 8×8 block was not that far behind, but it recorded a little bit slower. This is because each block does not utilize as much parallelism, therefore creating a greater number of total blocks. The 32×32 block size did not even run, therefore exceeding our hardware's limitations (1024 is the maximum number of threads per block). From the test we can conclude the 16×16 block size is the best choice.

Conceptual Question: For a 64×64 square tiled matrix multiplication, how many times is each element of the input matrices loaded from global memory? Assume 16×16 tiles.

- 3) For a 64×64 square tiled matrix, each element is loaded from global memory 4 times.

Conceptual Question: For a 64x64 square non-tiled matrix multiplication, how many times is each element of the input matrices loaded from global memory? Assume 16x16 tiles.

- 4) For a 64 x 64 square non-tiled matrix, each element is loaded from global memory 64 times.

Conceptual Question: For a 64x64 square tiled matrix multiplication, what is the reduction in total memory access for the entire kernel between the tiled and non-tiled implementation? Assume 16x16 tiles.

- 5) For a 64 x 64 square matrix, the memory load reduction from non-tiled to tiled implementation is 16 times.

Conceptual Question: Which tile size resulted in the least number of accesses to global memory? Which tile size resulted in the most number of accesses to global memory? What is the reasoning behind this observation?

- 6) A 32 x 32 tile size would result in the least amount of access from global memory to shared memory. This is because more data would be loaded into the tile before needing to reload new values into the tile. We could calculate more of the matrix before moving onto the next phase. A 8 x 8 tile size would result in the greatest amount of access from global memory to shared memory. This is because the smaller tile size would load less of the matrix into shared memory. This would mean we would need to frequently reload the data to calculate the matrix.

Conceptual Question: For a 64x64 square matrix multiplication, how many operations (multiply and add) are performed in total?

- 7) For a 64 x 64 matrix, there are 64 times 64 output elements. Each output element computes 64 multiplications and 64 additions. This results in the number 524,288.

Conceptual Question: Let's assume we're using an 8x8 tile for a tiled double-precision matrix multiplication. How many bytes of shared memory would each thread block require?

- 8) Each element in the double precision matrix is 8 bytes. There are a total of 64 elements and tiled matrix multiplication stores 2 tiles. Multiplying these values together results in 1024 bytes. Each thread block requires 1024 bytes of shared memory.